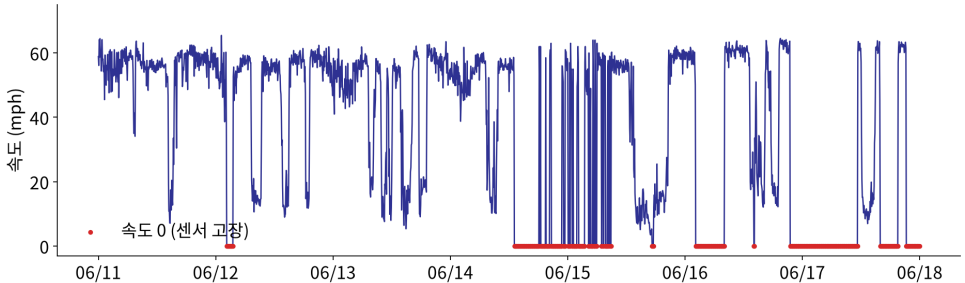
데이터 직접 다루기

더티 데이터 (결측치 · 중복 · 이상치) 와 정규화 · 표준화

Machine Learning and Deep Learning

KSA 2027-Spring
Sumin Han (suminhan@ksa.hs.kr)

오늘의 질문



LA 고속도로 센서 207개, 5분마다 잰 속도 3주. **평균 속도 50.8 mph**, 믿어도 될까?

METR-LA (Li et al., 2018): 2012-06-04 ∼ 06-27, 센서 #125 의 한 주

더티 데이터의 세 가지 모양

① 결측치	② 중복	③ 이상치
있어야 할 값이 없다	같은 기록이 여러 번	혼자 크게 튀는 값
Missing value	Duplicate	Outlier

모델을 고르기 전에, 데이터부터 본다

Garbage in, garbage out: 쓰레기를 넣으면 쓰레기가 나온다

노트북에서 열어 보면: METR-LA

In

```
raw = pd.read_parquet("data/metr_test.parquet", columns=["node_id", "t0_timestamp", "x_t+0_d0"])
raw["time"] = pd.to_datetime(raw["t0_timestamp"])
S_raw = raw.pivot(index="time", columns="node_id", values="x_t+0_d0")   # rows = time, columns = sensor
print(S_raw.shape); S_raw.stack().describe().round(2)
```

Out

```
(6850, 207)
count    1417950.00      mean     50.77      std     22.77
min            0.00      25%      45.38      50%     61.62
75%           65.89      max      70.00
```

140만 줄 → pivot → **시각 6,850 × 센서 207.** 이상한 점: **min 0**, **max 딱 70**

① 결측치: 0 mph 는 속도가 아니다

속도 0 인 칸	**12.1%**
0 을 그대로 두고 평균	**50.8 mph**
0 을 빈칸으로 바꾸고 평균	**57.8 mph**

0 은 “멈춰 있었다”가 아니라 “**재지 못했다**” (센서 고장 · 통신 끊김)

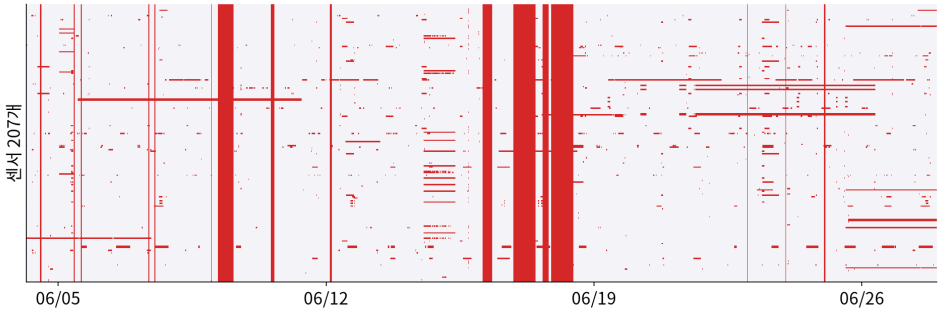
`S = S_raw.replace(0, np.nan)` 진짜 빈칸 (NaN) 으로 바꿔야 평균에서 빠진다

숫자인 척하는 결측치가 더 위험하다

적힌 값	컴퓨터가 보기에	
(빈칸), NaN	비었음	**눈에 띈다**
NA, ?	글자	계산하다 오류가 난다
0, -999	**진짜 숫자**	**오류 없이 평균에 섞인다**

속도 0 mph, 나이 -999세처럼 **그 분야에서 말이 안 되는 값**을 찾아서 빈칸으로 바꾼다

빈칸은 어디에, 얼마나 길게



세로 띠: 207개가 **한꺼번에** 빔
가로 줄: 한 센서가 오래 고장
점: 잠깐 끊김

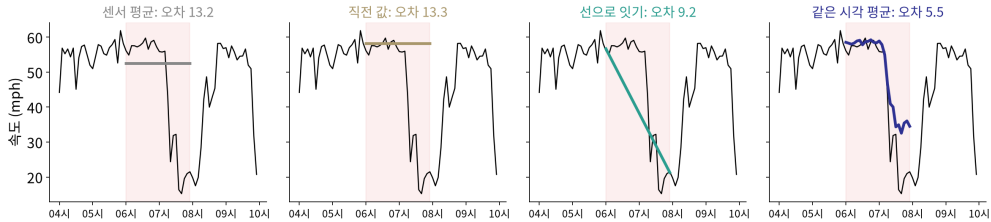
666 시각
1시간+ 434개, 하루+ 18개
5분짜리 349개

시스템 정전, 채울 수 없다
채우기 어렵다
앞뒤로 쉽게 채운다

fillna: 빈칸을 채우는 네 가지 방법

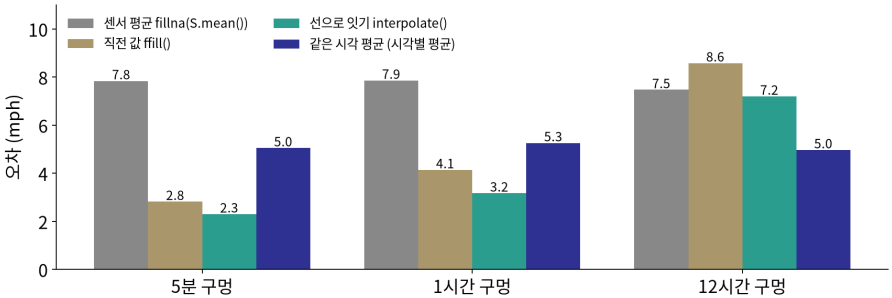
In

```
tod_mean = S.groupby(S.index.time).transform("mean")    # same sensor, same time of day
filled = {"sensor mean":       S.fillna(S.mean()),      # 1. that sensor's overall mean
          "ffill":             S.ffill(),               # 2. last value seen (forward fill)
          "interpolate":       S.interpolate(),         # 3. straight line between before and after
          "time-of-day mean": S.fillna(tod_mean)}       # 4. mean at the same time (e.g. 8:05) on other days
```



진짜 값 2시간 (06∼08시) 을 가리고 채워 보면: 출근길 정체를 따라가는 건 **같은 시각 평균**뿐

어느 방법이 맞을까: 진짜 값을 가려 놓고 채점



짧은 구멍은 **선으로 잇기**, 긴 구멍은 **같은 시각 평균**. **정답은 구멍 길이에 따라 다르다**

관측된 값을 무작위로 가리고 채운 뒤 가린 값과 비교 (5분 구멍 3,000개 · 1시간 300개 · 12시간 100개).
오차 = 평균 몇 mph 틀렸나

② 중복: 이 데이터에는 없다. 그래도 확인한다

In

```
print("duplicate (sensor, time) rows:", raw.duplicated(["node_id", "time"]).sum())
```

Out

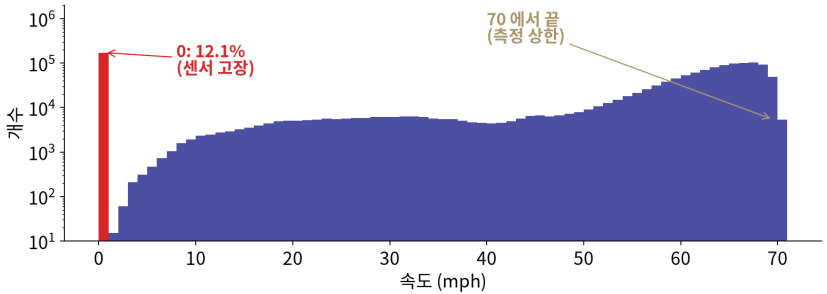
```
duplicate (sensor, time) rows: 0
```

같은 (센서, 시각) 이 두 번 있으면 그 시각이 **두 표**를 행사한다

파일을 합치거나 수집을 다시 돌리면 흔히 생긴다 → `pivot` 도 중복이 있으면 **오류**를 낸다

무엇을 "같은 기록"으로 볼지 (여기서는 센서 + 시각) 를 먼저 정한다

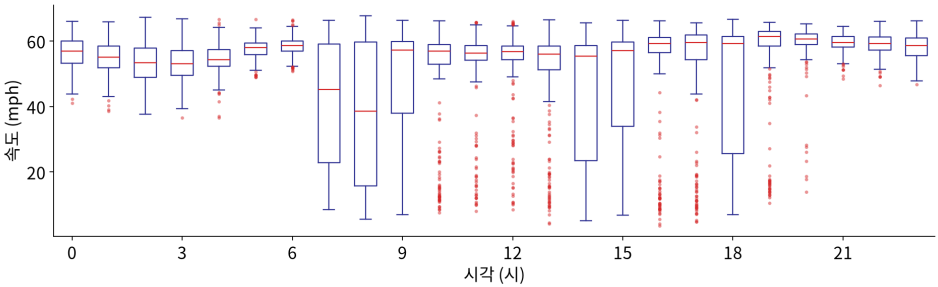
③ 이상치: 분포를 그려 본다



0 은 **결측치**, 70 에서 끊긴 벽은 **측정 상한** (70 넘으면 70 으로 적는다)

0 보다 크고 5 mph 미만인 값 **596개**: 고장일까, 진짜 정체일까?

그림으로 찾는다: 시각별 상자 그림



수염 밖의 점 = 이상치 **후보**. 출퇴근 (7∼9시, 15∼18시) 에 몰려 있다 → **진짜 정체**

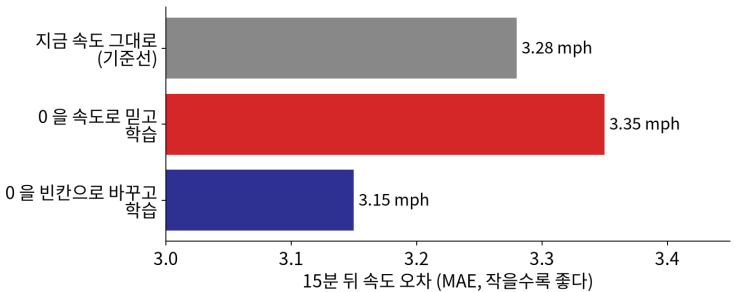
상자 = 가운데 절반 (25∼75%), 빨간 선 = 중앙값, 수염 = 상자 길이의 1.5배까지. 센서 #125, 0 은 빼고 그림

지울까, 채울까, 남길까

경우	METR-LA 에서	처리
숫자인 척하는 빈칸	속도 0 mph	**빈칸으로** (`replace(0, np.nan)`)
짧은 구멍	5분 ∼ 1시간 끊김	채운다 (`interpolate`)
긴 구멍	반나절 이상 고장	같은 시각 평균, 또는 **채우지 않고 표시**
전체 정전	207개가 동시에 0	그 시각은 **학습 · 채점에서 뺀다**
측정 상한	딱 70 mph	남긴다 (한계로 알아 둔다)
진짜일 수 있음	출근길 5 mph	**남긴다**

판단의 근거는 그 분야를 아는 지식 (도메인 지식)

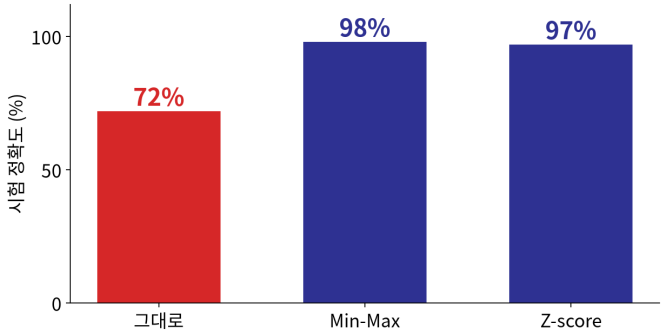
모델도 속는다



0 을 속도로 믿고 배운 모델은 **“지금 속도 그대로”보다도 못하다**

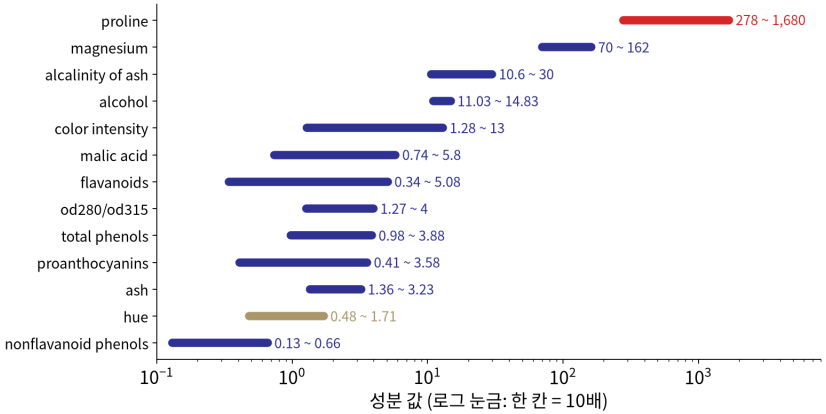
지난 1시간 (12칸) → 15분 뒤 속도, 선형회귀. 앞 70% 기간 학습, 뒤 30% 시험. 세 경우 모두 입력과 정답이 실제로 관측된 시험 줄 389,504개로 채점

두 번째: 값의 크기를 맞춘다



같은 신경망, 같은 와인 데이터. **성분 값을 그대로 넣으면 72%**, 정규화하면 **98%**

와인 한 병 = 성분 값 13가지, 범위가 제각각



프롤린 278 ~ 1,680, **색조 0.48 ~ 1.71**: 천 배 넘게 차이 난다

노트북에서 열어 보면: 와인 13성분

In

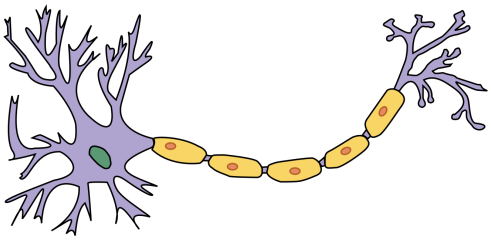
```
wine = load_wine(as_frame=True); df = wine.data
print(df.shape)
df.describe().T[["min", "max", "mean", "std"]].round(2)
```

Out

```
(178, 13)
                min      max    mean     std
alcohol       11.03    14.83   13.00    0.81
magnesium     70.00   162.00   99.74   14.28
hue            0.48     1.71    0.96    0.23
proline      278.00  1680.00  746.89  314.91
   ... (13 rows)
```

한 행 = 와인 한 병, 열 = 성분. **proline 의 std 315** vs **hue 0.23**

뉴런 하나가 하는 일



$$\text{output} = \tanh(w \cdot x + b)$$

x = 성분 값 13개 (데이터)

w = 뉴런이 **학습할 가중치** 13개

PyTorch `nn.Linear(13, 16)` 의 w 는 처음에
$-0.28 \sim 0.28$ 사이 작은 난수 ($\pm 1/\sqrt{13}$)

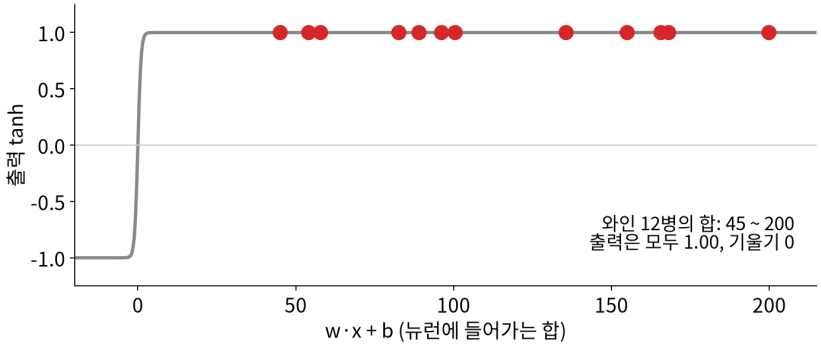
와인 1번 병을 그대로 넣으면

성분	값 x		가중치 w		$x \times w$
프롤린	**1,065**	$\times$	0.15	$=$	**162.15**
마그네슘	127	$\times$	0.05	$=$	6.56
알코올	14.23	$\times$	0.11	$=$	1.57
색조	1.04	$\times$	-0.16	$=$	-0.17
나머지 9개 $+ b$					-1.98
합 $w \cdot x + b$					**168.13**

$\tanh(168.13) = \mathbf{1.00}$ 프롤린 하나가 합을 다 차지한다

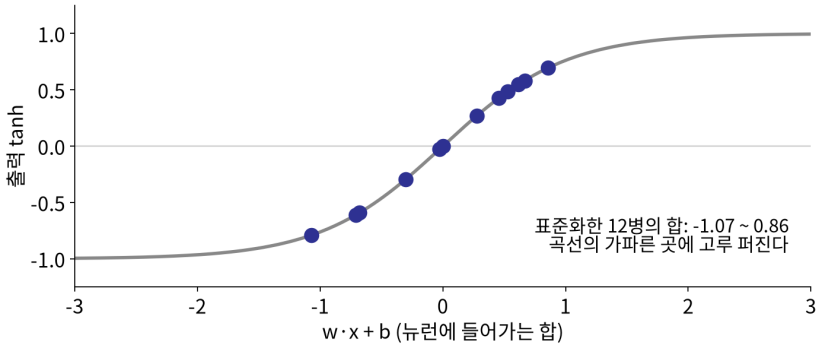
w 는 반올림해 적었고, 곱은 반올림 전 값으로 계산 (seed 0, 4번 뉴런)

12병 모두 출력 1.00: 기울기 0, 배우지 못한다



곡선이 평평한 곳 = **기울기 0**. 학습 전 뉴런 출력의 **98%**가 이렇게 막혀 있다

성분마다 분포를 맞추면



프롤린 1,065 → **1.01**. 합 **0.67**, 출력 **0.58**: 곡선의 가파른 곳에서 신호가 산다

방법 1. Min-Max 정규화: 0 ∼ 1 로

$$x' = \frac{x - \min}{\max - \min}$$

성분마다 따로: 가장 작은 값 → 0, 가장 큰 값 → 1

프롤린 1,065: $\frac{1065 - 278}{1680 - 278} = \frac{787}{1402} = \mathbf{0.56}$

쓰기 좋은 곳: 이미지 픽셀처럼 **범위가 정해진 값** (0 ∼ 255 → 0 ∼ 1)

방법 2. Z-score 표준화: 평균 0, 표준편차 1 로

$$z = \frac{x - \mu}{\sigma}$$

μ = 평균 (mean), σ = 표준편차 (standard deviation), 성분마다 따로

프롤린 1,065: $\frac{1065 - 747}{314} = \mathbf{1.01}$

"평균보다 **표준편차 1개만큼** 많다"

보통은 이것을 쓴다. 튀는 값 하나가 있어도 Min-Max 보다 덜 흔들린다

평균 · 표준편차는 학습 데이터로만 구한다

단계	할 일	
학습 데이터	μ, σ (또는 min, max) 를 **구한다**	`fit`
학습 데이터	그 값으로 바꾼다	`transform`
시험 · 새 데이터	**같은** μ, σ 로 바꾼다 (새로 구하지 않는다)	`transform`

시험 데이터로 평균을 구하면 **시험 문제를 미리 엿본 것** (데이터 누수)

회귀에서 정답 y 도 표준화했다면, 예측값은 마지막에 $\hat{y} = \hat{z}\,\sigma_y + \mu_y$ 로 원래 단위로 되돌린다

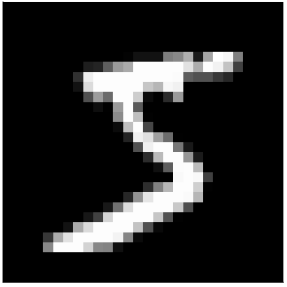
데이터마다: “무엇마다” 맞추나

데이터	모양	무엇마다	보통 하는 일
표 (tabular)	(행, 열)	**열(성분)마다**	Z-score
흑백 이미지	(H, W)	이미지 전체	$\div 255$, 그다음 평균 · 표준편차
컬러 이미지	(H, W, 3)	**채널(R, G, B)마다**	$\div 255$, 채널별 평균 · 표준편차
언어	단어의 줄	글자 표기	표기 맞추기, 번호는 **그대로**
시계열	(시간,)	**과거만**, 또는 창마다	학습 기간 Z-score, 창마다 Z-score
소리	(샘플,)	녹음마다	$\div 32768$, 음량 맞추기, dB
비디오	(T, H, W, 3)	채널마다, **영상 전체로**	이미지와 같게, 프레임마다 따로는 X

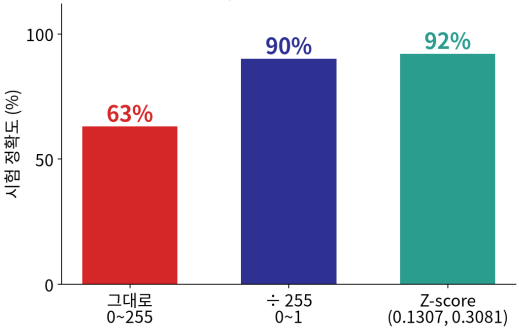
먼저 **데이터를 열어서** 모양 · 자료형 · 최솟값 · 최댓값을 본다

흑백 이미지: 픽셀 0 ∼ 255 를 그대로 넣으면

28 × 28 픽셀, 값 0 ~ 255 (uint8)



MNIST, 같은 신경망 · 같은 학습



MNIST: $\div 255$ 만 해도 **63%** $\rightarrow$ **90%**. 그대로면 학습 전 뉴런 **95%**가 막힌다

학습 1만 장 · 시험 1만 장, MLP 784-64-10 (tanh), SGD 0.1, 3에폭, 시드 3개 평균. 0.1307, 0.3081 = 학습 6만 장의 픽셀 평균 · 표준편차

노트북에서 열어 보면: MNIST

In

```
tr = datasets.MNIST("data", train=True, download=True)
X_train = tr.data.numpy()
look("MNIST train", X_train)
look("MNIST / 255", to_unit(X_train))
```

Out

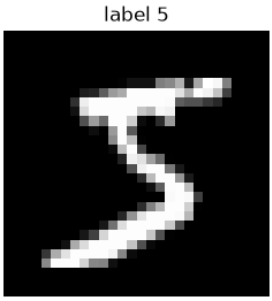
```
MNIST train shape=(60000,28,28) dtype=uint8   min=0 max=255 mean=33.32 std=78.57
MNIST / 255 shape=(60000,28,28) dtype=float32 min=0 max=1 mean=0.1307 std=0.3081
```

In

```
pixels = X_train[0, 14, 8:14]
print(pixels - np.uint8(50))
```

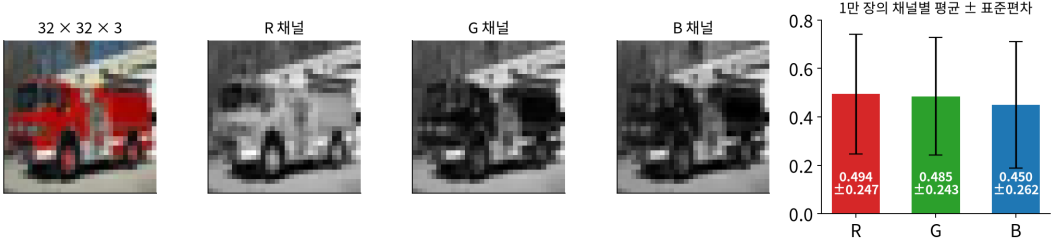
Out

```
[206 206 206 206 206  31]      # 0 - 50 = 206 !
```



uint8 은 0 아래로 못 간다
→ 먼저 실수형으로

컬러 이미지: 채널(R, G, B)마다 따로



CIFAR-10: 평균 R **0.494**, G **0.485**, B **0.450**. 채널마다 빼고 나눈다

사진은 (H, W, C), PyTorch 는 **(C, H, W)**: 축 순서부터 확인

남이 학습시킨 모델은 그 모델의 값을 그대로 (ImageNet: 0.485, 0.456, 0.406)

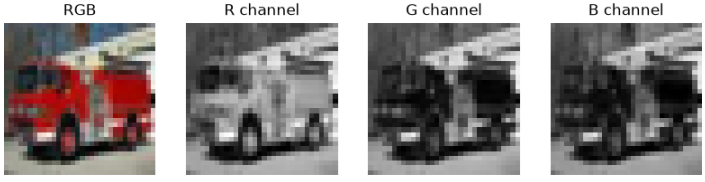
노트북에서 열어 보면: CIFAR-10

In

```
cifar = pd.read_parquet("data/c10_test.parquet")
C = np.stack([np.array(Image.open(io.BytesIO(b["bytes"])).convert("RGB")) for b in cifar["img"]])
look("CIFAR-10", C)
C01 = C / 255
print("mean (R, G, B):", C01.mean(axis=(0, 1, 2)).round(3))
```

Out

```
CIFAR-10   shape=(10000, 32, 32, 3) dtype=uint8  min=0  max=255  mean=121.5  std=64.06
mean (R, G, B): [0.494 0.485 0.45 ]
```



모양 = (장, 높이, 너비, **채널**): 채널 축만 남기고 평균 → 숫자 3개

언어: 숫자 크기가 아니라 표기를 맞춘다

① 표기 맞추기

FREE, Free, free! $\rightarrow$ free
SMS 5,574통의 단어 종류 **15,733** $\rightarrow$ **8,713**

② 단어 번호는 크기가 아니다

free $= 3373$, call $= 1828$: 3373 이 "더 큰" 단어는 아니다
$\rightarrow$ 번호를 표준화하지 않는다. 임베딩 벡터로 바꾼다
(nn.Embedding 은 처음에 평균 0, 표준편차 1 난수)

③ 단어 세기는 길이로 나누기

1단어 문자와 171단어 문자: 긴 글일수록 숫자가 커진다

노트북에서 열어 보면: SMS 문자

In

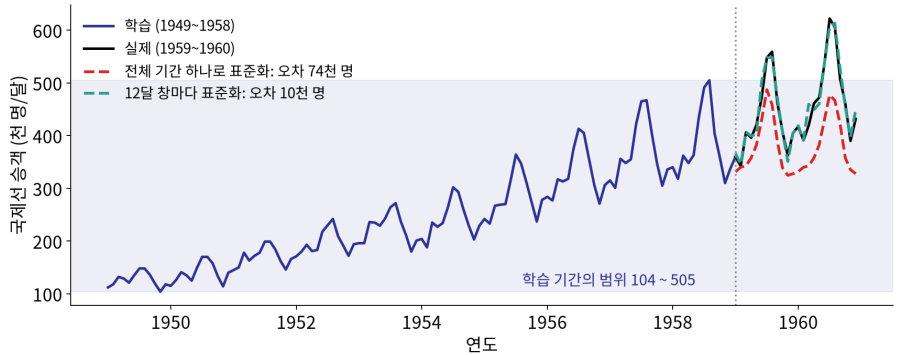
```
df = pd.read_csv(z.open("SMSSpamCollection"), sep="\t", names=["label", "text"], quoting=3)
print(df.shape); print(df.text.head(2).tolist())
raw_words = " ".join(df.text).split()
print("word types:", len(set(raw_words)))
print([w for w in set(raw_words) if w.lower().strip("!.,?") == "free"])
```

Out

```
(5574, 2)
['Go until jurong point, crazy.. Available only in bugis n great world la e buffet...',
 'Ok lar... Joking wif u oni...']
word types: 15733
['free.', 'free..', 'Free!', 'FREE!', 'FREE.', 'free...', 'free,', 'free!', 'Free', ...] (13)
```

한 행 = 문자 한 통 (label: ham / spam). **"free" 가 13가지**로 따로 센다 → 표기 맞추면 **8,713**

시계열: 미래는 과거의 범위를 넘는다



학습 최대 505 → 시험 **622** (Min-Max **1.29**) 창마다 표준화: 오차 **74** → **10**천 명

국제선 항공 승객 1949∼1960, 앞 10년 학습 · 뒤 2년 시험, 지난 12달 → 다음 달, k-최근접 이웃 ($k = 3$)

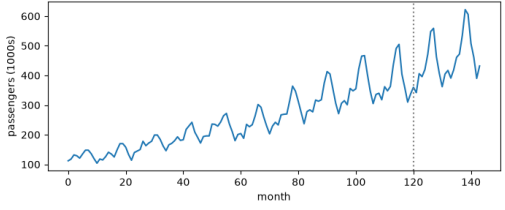
노트북에서 열어 보면: 항공 승객

In

```
air = pd.read_csv("data/air.csv")
series = air["Passengers"].values.astype(float)
look("train 1949-58", series[:120])
look("test 1959-60", series[120:])
```

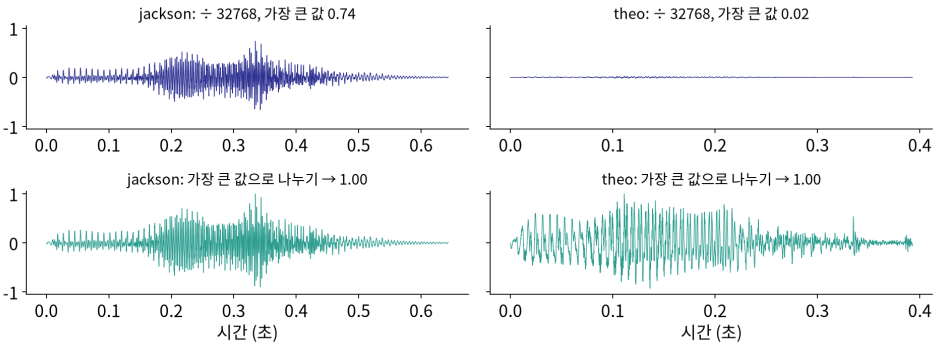
Out

```
train 1949-58  shape=(120,)  dtype=float64  min=104  max=505  mean=245.9  std=94.55
test 1959-60   shape=(24,)   dtype=float64  min=342  max=622  mean=452.2  std=74.68
```



시험 기간 최댓값 **622** > 학습 기간 최댓값 505

소리: 녹음마다 음량이 다르다



같은 “zero”, 음량 **37배** 차이. int16 $\div 32768 \to -1 \sim 1$, 그다음 **녹음마다** 맞춘다

Free Spoken Digit Dataset, 8,000 Hz. 가장 큰 값으로 나누기 = 피크 정규화 (녹음마다 표준편차로 나누는 방법도 쓴다)

노트북에서 열어 보면: 숫자 말소리

In

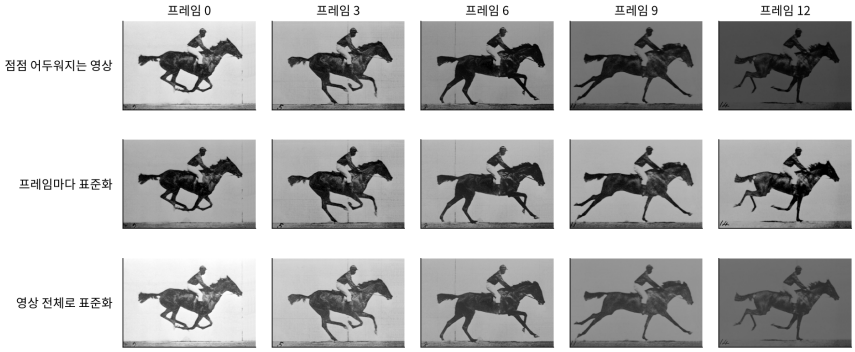
```
for s in SPEAKERS:
    rate, w = wavfile.read(f"data/0_{s}_0.wav")
    look(s, w)
for s in SPEAKERS:
    print(f"{s:9s} peak = {np.abs(to_float(waves[s])).max():.2f}")   # to_float: / 32768
```

Out

```
jackson   shape=(5148,)  dtype=int16  min=-2.166e+04  max=2.416e+04  mean=-0.2374  std=4482
theo      shape=(3142,)  dtype=int16  min=-606        max=655        mean=-0.2772  std=177
   ... (6 speakers)
jackson   peak = 0.74
theo      peak = 0.02          loudest / quietest: 37
```

한 행 = 1/8000초마다 잰 숫자. 같은 “zero” 인데 길이도 (5148 vs 3142) 음량도 (**37배**) 다르다

비디오: 이미지 × 시간, 영상 전체로 맞춘다



(T, H, W, C) = (15, 200, 300, 3). 프레임마다 표준화하면 **어두워지는 변화가 사라진다**

Muybridge, The Horse in Motion (1878). 실습에서는 일부러 점점 어둡게 만든 영상으로 비교

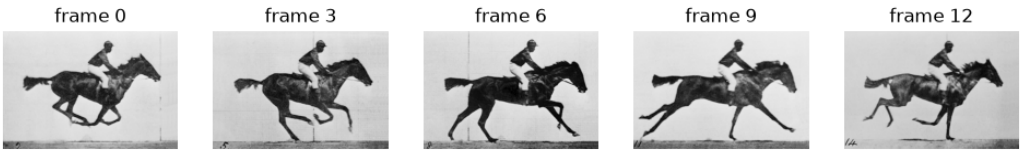
노트북에서 열어 보면: 달리는 말 GIF

In

```
gif = Image.open("data/horse.gif")
video = np.stack([np.array(f.convert("RGB")) for f in ImageSequence.Iterator(gif)])
look("video", video)
```

Out

```
video   shape=(15, 200, 300, 3)  dtype=uint8  min=0  max=245  mean=177.4  std=68.08
```



모양 = (**프레임**, 높이, 너비, 채널): 이미지가 시간 순서로 15장

문제풀이

1. 센서 하나의 속도 `[60, 0, 0, 62, 64]` (mph). 그대로 평균은? 0 을 빈칸으로 보고 빼면? `interpolate` 로 채우면 두 빈칸은?
2. 다음 값은 오류일까, 진짜일까? 무엇을 더 알아야 할까? (a) 새벽 3시 속도 3 mph (b) 오후 6시 속도 3 mph
3. 8명의 하루 공부 시간 `[2, 4, 4, 4, 5, 5, 7, 9]`. 평균과 표준편차(n 으로 나눔)는? 9시간의 Z-score 는? 5시간의 Min-Max 값은?
4. 학습 데이터의 키 최솟값 150, 최댓값 190 으로 Min-Max 를 만들었다. 시험 데이터에 키 200 이 나오면 값은? 시험 데이터로 min, max 를 다시 구해야 할까?
5. `np.uint8(10) - np.uint8(20)` 은 얼마일까? 컬러 이미지 `X.shape == (10000, 32, 32, 3)` 의 채널별 평균은 `X.mean(axis=?)` ?

실습: 데이터를 직접 열어 보기

1. 더티 데이터: LA 교통 센서 METR-LA, 0 mph → 빈칸, `fillna` 네 가지 채점, 0 을 믿은 모델 **3.3**
2. 표: 와인 13성분, Min-Max · Z-score 구현 → **72%** vs **98%**
3. 흑백 이미지: MNIST uint8 함정, $\div 255$, 평균 · 표준편차
4. 컬러 이미지: CIFAR-10 채널별 평균 · 표준편차, (H, W, C) → (C, H, W)
5. 언어: SMS 표기 맞추기 (15,733 → 8,713), 단어 세기 길이로 나누기
6. 시계열: 항공 승객, 학습 기간으로만 vs 창마다 표준화 (**74** → **10**)
7. 소리: 숫자 말소리 6명, $\div 32768$, 음량 맞추기, dB 스펙트로그램
8. 비디오: 달리는 말 GIF, 프레임마다 vs 영상 전체

데이터는 첫 셀에서 내려받기 (MNIST · CIFAR-10 은 수십 MB)

▶ Colab에서 실습 노트북 열기

정리

결측치는 **숫자인 척**한다 (0 mph). 채우는 법은 **구멍 길이**로, 이상치 판단은 사람이

값의 크기가 제각각이면 뉴런이 **막힌다** (tanh 출력 1.00, 기울기 0)

Min-Max $\frac{x - \min}{\max - \min}$ Z-score $\frac{x - \mu}{\sigma}$ 통계는 **학습 데이터로만**

무엇마다 맞출지는 데이터가 정한다: 열 · 채널 · 녹음 · 창 · 영상 전체

어떤 데이터든 먼저 **열어서** 모양 · 자료형 · 최솟값 · 최댓값을 본다

이미지 · 데이터 출처

- 뉴런 그림: Wikimedia Commons “Neuron.svg” (Dhp1080, CC BY-SA 3.0), 라벨 선을 지워 사용
- 달리는 말: Eadweard Muybridge, The Horse in Motion (1878), 퍼블릭 도메인 (Wikimedia Commons)
- 교통 속도: METR-LA, Y. Li, R. Yu, C. Shahabi, Y. Liu, “Diffusion Convolutional Recurrent Neural Network” (ICLR 2018), Hugging Face witgaw/METR-LA
  와인: UCI Wine (Aeberhard & Forina, 1991) 손글씨 숫자: MNIST (LeCun, Cortes, Burges)
- 컬러 사진: CIFAR-10, A. Krizhevsky (2009) “Learning Multiple Layers of Features from Tiny Images”
- 문자 메시지: UCI SMS Spam Collection (Almeida & Hidalgo, 2011)
- 항공 승객: Box & Jenkins, Time Series Analysis (1976), 1949∼1960 국제선 월별 승객
- 말소리: Free Spoken Digit Dataset (Z. Jackson 외, CC BY-SA 4.0)